

AEM Multifield

How It Works · When to Use · Where to Use

What is a Multifield?

A Multifield lets content authors add, remove, and reorder multiple rows of data inside a component dialog — without any code changes. Think of it as a dynamic list builder for authors.

■ Example: 'I want the author to add as many navigation links as they want' → Use Multifield

Two Types — Simple vs Composite

SIMPLE	COMPOSITE
One field per row → List of URLs → List of keywords → List of tags <code>composite = false</code>	Multiple fields per row → Link Text + URL + Target → Image + Alt + Caption → Title + Desc + CTA <code>composite = true</code>

Decision Flow

Ask yourself one question before choosing:

Do you need more than one field per row?

YES → Composite Multifield (`composite=true`)

NO → Simple Multifield (`composite=false`)

Real-World Use Cases

Use Case	Type
Author adds multiple page links	Simple
Author adds nav items (label + url + icon)	Composite

Author adds a list of tags / keywords	Simple
Author builds card list (title + desc + image)	Composite
Author adds multiple DAM image paths	Simple
Author adds testimonials (quote + author + photo)	Composite
Author adds allowed file types	Simple

How Data is Stored in JCR

Simple → stored as a multi-value property:

```
/content/mypage/jcr:content/mycomponent
links = ["https://a.com", "https://b.com"] ← String[]
```

Composite → stored as child nodes (one node per row):

```
/content/mypage/jcr:content/mycomponent/links
item0/
linkText = "Google"
linkUrl = "https://google.com"
linkTarget = "_blank"
item1/
linkText = "Adobe"
linkUrl = "https://adobe.com"
linkTarget = "_self"
```

End-to-End Data Flow

- 1 Author fills dialog
- 2 AEM saves to JCR
- 3 Sling Model reads JCR
- 4 HTL renders it

Simple path:

Dialog → String[] property in JCR → @ValueMapValue String[] in Sling Model → loop in HTL

Composite path:

Dialog → child nodes (item0, item1...) in JCR → @ChildResource + iterate children → loop in HTL

Reading in Sling Models

Simple Multifield

```
@Model(adaptables = Resource.class)
public class MyModel {
    @ValueMapValue
    private String[] links; // direct multi-value property

    public String[] getLinks() { return links; }
}
```

Composite Multifield

```
@Model(adaptables = Resource.class)
public class MyModel {
    @ChildResource
    private Resource links; // the parent node

    public List getLinkItems() {
        List items = new ArrayList<>();
        if (links != null) {
            for (Resource child : links.getChildren()) {
                ValueMap vm = child.getValueMap();
                items.add(new LinkItem(
                    vm.get("linkText", String.class),
                    vm.get("linkUrl", String.class),
                    vm.get("linkTarget", "_self")
                ));
            }
        }
        return items;
    }
}
```

Quick Reference

Property	Simple	Composite
Fields per row	1	2 or more
JCR storage	Multi-value property	Child nodes

Sling Model	String[]	Iterate Resource children
composite=	false	true
Author experience	Single input per row	Group of fields per row

Common Gotchas

■ composite=true is mandatory for grouped fields

Without it, AEM only saves the last field's value and overwrites everything else. Always set it explicitly.

■ Don't rely on node names

AEM auto-generates item0, item1... for composite rows. Never hardcode node names — always iterate the children.

■ Order is preserved

Child nodes maintain insertion order in JCR, so the order the author sets in the dialog is the order you get in code.

■ Nesting Multifields

Nesting one multifield inside another is not supported out of the box — it needs custom Granite UI or JS workarounds.